Accelerating Heterogeneous Tensor Parallelism via Flexible Workload Control

Zhigang Wang, Xu Zhang, Ning Wang, Chuanfei Xu, Yu Gu, Hui Lu, Dawei Zhao, and Zhihong Tian, *Senior Member, IEEE*

***Abstract*—Transformer-based foundation models are becoming deeper and larger. For fast training, their billions of parameters (tensors) are split onto parallel tasks running on many modern yet expensive accelerators. To amortize the huge hardware investments, it is cost-effective to share the aggregated resources among multi-tenants. However, resource contention yields the heavy straggling problem. Existing works feature contributions for the traditional data parallelism. They cannot work well for the new tensor parallelism, due to the dependency among split tensors. This paper is the first attempt on accelerating heterogeneous tensor parallelism. We summarize specific challenges, including the very frequent synchronizations and the heavy tensor computation workloads. Our solution is to resize dimensions of parameters on demand, to quickly and dynamically balance workloads. The accuracy loss is reduced through priority resizing. We also migrate workloads between tasks, without any loss of accuracy. The most efficient communication primitives are selected and then scheduled in a non-redundant manner, to reduce the runtime latency. Our final hybrid solution is built on top of resizing and migration. By studying the tradeoff between accuracy and efficiency, it can smartly hit the "sweet spot". Extensive experiments validate the effectiveness of our proposals.**

Received 22 April 2025; revised 22 November 2025; accepted 26 December 2025. Date of publication 23 January 2026; date of current version 12 May 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62572139, Grant U25A20424, Grant 62572108, Grant 62272121, Grant U2436208, and Grant 62372129, in part by the Key R&D Program of Shandong under Grant 2025CXPT082, in part by the National Key Research and Development Plan under Grant 2023YFB3107300, in part by Guangdong S&T Program under Grant 2024B0101010002, in part by the Project of Guangdong Key Laboratory of Industrial Control System Security under Grant 2024B1212020010, in part by Science and Technology Projects of Xizang Autonomous Region under Grant XZ202501ZY0026, in part by the Major Key Project of PCL under Grant PCL2024A05, and in part by the Key Laboratory of Computing Power Network and Information Security, Ministry of Education under Grant 2024ZD016. Recommended for acceptance by D. B. Rawat. *(Zhigang Wang and Chuanfei Xu are co-first authors.) (Corresponding authors: Dawei Zhao; Ning Wang.)*

Zhigang Wang, Ning Wang, Hui Lu, and Zhihong Tian are with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 510006, China, and also with the Guangdong Key Laboratory of Industrial Control System Security, Huangpu Research School of Guangzhou University, Guangzhou 510006, China (e-mail: wangzhiganglab@gmail.com; wangninggzu@gmail.com; luhui@gzhu.edu.cn; tianzhihong@gzhu.edu.cn).

Xu Zhang is with the Faculty of Information Science and Engineering, Ocean University of China, Qingdao 266100, China (e-mail: zhangxu1126@stu.ouc.edu.cn).

Chuanfei Xu is with the Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen 518000, China (e-mail: chuanfeixu@126.com).

Yu Gu is with the School of Computer Science and Engineering, Northeastern University, Shenyang 110819, China (e-mail: guyu@mail.neu.edu.cn).

Dawei Zhao is with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Qilu University of Technology (Shandong Academy of Sciences), Jinan 250014, China (e-mail: zhaodw@sdas.org).

Digital Object Identifier 10.1109/TBDATA.2026.3657302

***Index Terms*—Tensor parallelism, multi-tenant sharing, dynamic workload balancing, heterogeneous computations.**

I. Introduction

CURRENTLY, hundreds of transformed-based foundation models have been released, such as Llama2 for NLP [1] and Vision Transformers (ViT) for image understanding [2]. To effectively train these models with billions of parameters (tensors), many modern yet expensive accelerators (e.g., GPUs) are required to provide enough compute and memory resources. Existing parallelism solutions mainly include data parallelism [3] and model parallelism [4] where input samples and model tensors are respectively split among tasks. **Tensor Parallelism** [5] is the representative of the latter, which is tailored for foundation models due to the prominent memory scalability.

A transformer architecture typically consists of tens or even hundreds of stacked network layers. Tensor parallelism needs to *frequently synchronize all tasks* between any two consecutive layers, so as to correctly collect global data. The overall training efficiency is then very sensitive to the speed of each task. Meanwhile, there exists a lot of *compute-intensive matrix multiplications* in tensor computations. Thus, a task proceeds quickly if it can acquire many compute resources.

On the other hand, the numerous downstream applications and the hyper-parameter optimization yield the ever-growing demands of running different training jobs. It is prohibitively expensive to allocate all resources for so many jobs one by one, due to the huge hardware investments and the low resource utilization. Different from the dedicated allocation, it is cost-effective to share resources among jobs issued by multi-tenants. However, under the widely used best-effort or burst-share sharing policy, the distribution of available resources among accelerators can be skewed. Tasks belonging to the same job might proceed at different speeds, leading to the heavy and dynamic straggling problem.

We are aware that the straggling problem has been studied by many researchers. Existing techniques have made significant advancements but most are for data parallelism [6], [7], [8], [9], [10], which cannot be extended to our focused tensor parallelism. Recently some pioneers explore the heterogeneous tensor parallelism. However, they are tailored for either static stragglers [11], [12] or computation within layers [13]. Thus, there is an imperative need for efficient tensor parallelism in dynamic heterogeneous environments.

We first propose to temporarily **resize** matrices on stragglers on demand. It prunes columns or rows (dual) during computations for workload reduction; and recovers dimensions of outputs by imputing zeros, to normally collect data in synchronization. We prioritize matrix elements by their importance, to guide the selection of pruned data. That reduces the error caused by estimation and imputation in pruning, and then mitigates the accuracy loss. Besides, by indexing pruned and recovered dimensions, we can exactly match gradients with parameters for correct tensor refinement.

As another alternative balancing approach, we propose to **migrate** workloads from stragglers to fast tasks, and tackle the runtime latency head on. On one hand, we select high-throughput broadcasting and reducing primitives to address the single-point bottleneck during communications. On the other hand, we transform global communications into local variants, and then merge the latter into normal data collections, to eliminate redundant communications.

Resizing and migration have different favorites in runtime efficiency and model accuracy. However, when heavy heterogeneity happens, neither of them works well, because a large amount of errors (data) are generated (migrated). That motivates us to design a **semi**-migration solution where the two approaches are smoothly combined. End-users can specify the boundary of each approach to satisfy their favorite on accuracy loss and runtime latency.

The major contributions are summarized below.

- Proposing a matrix-resizing approach to dynamically balance workloads and normally perform synchronizations, with negligible runtime penalty. It employs indexing and priority techniques to respectively guarantee correct parameter refinement and mitigate the accuracy loss.
- Proposing another lightweight migration approach to eliminate accuracy penalty. It decreases the runtime latency via tree-based broadcasting/reducing and merging redundant communications.
- Building a hybrid solution *SEMI*-migration on top of the two approaches. It smartly runs the reasonable combination in different scenarios, by a cost-benefit analysis.
- Conducting extensive experiments on the newly released Colossal-AI [14] platform, using foundation models with billions of parameters. Our proposals improve the training efficiency by 22% with only 1.48% accuracy loss, in real heterogeneous environments. We also find that the resizing approach can accelerate training in homogeneous environments with negligible accuracy loss.

Below, Section II overviews tensor parallelism and summarizes challenges in heterogeneous environments. Sections III and IV present our solutions. Section V gives experiment results. Section VI summarizes related works and Section VII concludes this paper.

II. Preliminaries

Tensor parallelism is widely used in parallel transformer training, but it has frequent synchronizations between its compute intensive matrix computations. That yields resource underutilization in the heterogeneous environments.

A. Training a Transformer with Tensor Parallelism

A transformer model consists of many parameters (tensors) stored in stacked transformer layers with multi-head attention (MHA) and feedforward networks (FFN). Currently, two policies to split tensors reduce the storage burden. One is pipeline parallelism, where the bubble waiting cost lowers resource utilization. The other is **Tensor Parallelism** (TP), which is widely used in current platforms. It distributes parameters within every layer among tasks to fully utilize compute power but needs to frequently synchronize tasks to collect partial results of split tensors.

During training, the total input samples are divided into several batches and then scheduled in a round-robin manner. A batch of size $\mathbb{B}$ yields an iteration with Forward Propagation (FP) for extracting features and Backward Propagation (BP) for refining parameters. Given an iteration, the core operations in a layer are matrix multiplications between input X and weight parameter W. Take FFN with two linear weights W^A and W^B as an example. Under the classic 1D tensor parallelism [5], weights are split onto $\mathbb{T}$ tasks respectively in column- and row-wise manners,

$$W^A = \left[W_1^A : ... : W_{\mathbb{T}}^A\right], W^B = \left[W_1^B; ...; W_{\mathbb{T}}^B\right].$$

Matrix operators provided by mainstream platforms [14], [15], typically use the transpose of W^A/W^B. During FP, X is duplicated across tasks to extract partial features Y_i and Z_i in parallel, as shown in (1).

$$Y_i = X\left(W_i^A\right)^T, Z_i = Y_i\left(W_i^B\right)^T, i \in \{1, 2, ..., \mathbb{T}\} \quad (1)$$

The entire output $Z = \sum Z_i$ is then available via *all-reduce*, and is fed as input for the next layer. In contrast, BP computes gradients and then refines weights in a reverse direction, shown in (2). Similarly, the gradient input ∇Z received from the next layer is replicated; while the output $\nabla X = \sum X_i$ is derived via another *all-reduce* and fed into the previous layer. Resembling FFN, multi-head attention also runs *all-reduce* twice for synchronization.

$$\begin{cases} \nabla Y_i = \nabla Z * W_i^B, \nabla W_i^B = (\nabla Z)^T * Y_i, \\ \nabla X_i = \nabla Y_i * W_i^A, \nabla W_i^A = (\nabla Y_i)^T * X, \end{cases} \quad (2)$$

Besides, matrix multiplication is compute-intensive. Assume that the length of the sequence of input samples is $\mathbb{S}$, and the size of hidden layers is $\mathbb{H}$. Only for Z in FP of FFN, the element-wise multiplication complexity in one iteration is up to $\Omega(\mathbb{B} \times \mathbb{S} \times \mathbb{H}^2)$.

Take the ViT [2] model as an example. Assume that it has $\mathbb{H} = 2048$ and $\mathbb{D} = 24$ (the number of stacked transformer layers). The total number of parameters is up to 1.2 billion. Each iteration requires 96 *all-reduce* operations to synchronize tasks. Assume that the number of epochs is 150 and each further consists of 10,000 iterations. The training process requires up to *144 million synchronizations*. On the other hand, assume that $\mathbb{B} = 64$ and $\mathbb{S} = 65$. When computing Z in each iteration, there exist *419 billion floating-point operations*.

B. Heterogeneity in Multi-tenants Environments

Till now, many AI clusters and super-servers have been built by aggregating tens and even thousands of accelerators, to efficiently train foundation models. The communication bottleneck is mitigated through expensive high-speed NVLink and/or InfiniBand connections.[1] This cluster or server is usually shared by multi-tenants, to amortize the huge investments. For example, Meta spends billions of dollars on its private cluster RSC to handle up to 35,000 jobs a day [16]. Renting a cloud server can ease the financial burden, but the reduced cost is still out of reach for most scientists, if it is used in a dedicated manner. For example, Amazon EC2 charges $2,400 per day for its P5 instance with 8 GPUs [17]. Moreover, the deployment of pre-trained foundation models has now entered the domain-application phase, where numerous modest enterprises and research institutions are engaged in fine-tuning domain-specific models. Despite facing complex real-world requirements, these organizations typically operate under constrained computational resources—often limited to just a few high-performance servers. The resource-sharing technology also emerges as a solution, enabling fine-grained control over resource allocation, dynamically adjusting memory and computing power as needed.

Currently, many tools such as Kubernetes are used to share resources among multi-tenants. These platforms typically employ dynamic resource allocation strategies, such as burst-sharing mechanisms, best-effort allocation and their variants [18], [19], to improve the user-experience and maximize hardware utilization [20]. However, the resource contention phenomenon still exists, resulting in a highly heterogeneous training environment. Below we list two representative cases.

CASE-I: preempting resources can significantly impact task performance, particularly in multi-tenant environments where short-burst requests interfere with sustained heavy workloads. For instance, a job of training a billion-parameter model, J_i, typically monopolizes all available servers, with parallel tasks spanning weeks. During this extended execution, efficient resource management becomes critical—especially when high-priority short-burst requests like J_k arise. Despite the lightweight nature of J_k, J_k inevitably preempts memory and compute resources allocated to ongoing tasks in J_i. As a result, the affected tasks suffer performance degradation, leading to inconsistent execution speeds across the workload.

CASE-II: dynamically releasing resources can introduce similar performance imbalances. When a task from job J_i completes, its freed resources are evenly redistributed among remaining tasks on the same accelerator A_x. This accelerates concurrent tasks of another job J_k on A_x. This strategy has been widely adopted by cloud providers like NVIDIA (vGPU), Alibaba (cGPU), and Tencent (qGPU). Note that under fair scheduling policies, tasks from multiple jobs may be arbitrarily distributed across accelerators, compounded by unpredictable task lifetimes. With the completion of some jobs, for tasks belonging to the same ongoing job but running on different accelerators, the pre-set even distribution of available resources becomes skewed. For example, tasks of J_k on another accelerator A_y cannot enjoy this bonus if jobs on A_y are still in progress. This heterogeneity in resource distribution leads to slower progression for disadvantaged tasks, ultimately turning them into stragglers.

[1] Ref. [5] reports that only 23% of training time is spent on communications, for tensor parallelism on 8 NVIDIA V100 GPUs connected by NVLink.

C. Performance Challenges

Section II-A has revealed that tensor parallelism exhibits two key characteristics, i.e., frequent synchronization requirements and computationally intensive workloads. Both factors are highly sensitive to dynamic resource fluctuations. In heterogeneous computing environments, even minor variations in resource availability can substantially impact computational performance. This imbalance creates synchronization bottlenecks, where fast tasks are forced to idle, to wait for slow, straggling tasks.

To quantify the performance impact, we have conducted experiments using an 8-GPU server to run a 1-billion parameter Vision Transformer (ViT) on the CIFAR-10 dataset (experimental setup detailed in Section V-A). To simulate multi-tenant GPU contention as observed in Tencent's production environments [21], we concurrently submit high-priority ResNet-34 training jobs. Each job has a single task running on a GPU card and randomly terminates. This configuration creates two distinct scenarios, i.e., some GPUs run both ResNet-34 and ViT tasks, while others are dedicated to ViT execution only. Our measurements reveal significant performance disparity. The times of completing a ViT epoch are 781 secs (co-located case) versus 373 secs (dedicated case). Due to the synchronization requirements of tensor parallelism, this speed differential propagates through the entire system. The slowest ViT task determines the overall epoch completion time (781 seconds), forcing faster GPUs to remain idle while waiting for stragglers. This is an evident waste of computational resources. This paper thereby focuses on these straggler-induced inefficiencies and gives systematical solutions.

III. Balancing Workloads by Resizing Matrices

This section describes matrix resizing for dynamic workload balancing. We give the pruning and recovering policies and then present our priority design for accuracy enhancement.

A. Dual-pruning and Recovering Dimensions

Clearly, if workloads on stragglers are reduced on demand, then all tasks can be synchronized without any waiting costs. We achieve this goal by dynamically resizing matrices, consisting of pruning and recovering procedures.

Pruning for Workload Reduction: Given a matrix, rows or columns are selected and pruned, respectively tailored for the row-splitting and column-splitting patterns used in tensor parallelism. We thereby call this method as dual-pruning. We uniformly denote by γ the pruning ratio of the number of pruned dimensions to the number of original ones. Fig. 1 uses the column-splitting with respect to W^A as an example.

During FP, our main idea is to estimate the value of each element in the output matrix, using the pruned low-rank factor

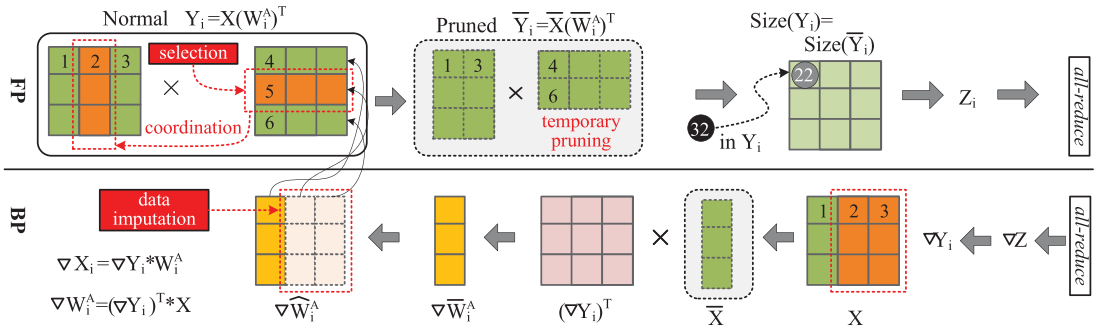


Fig. 1. Matrix pruning and recovering process.

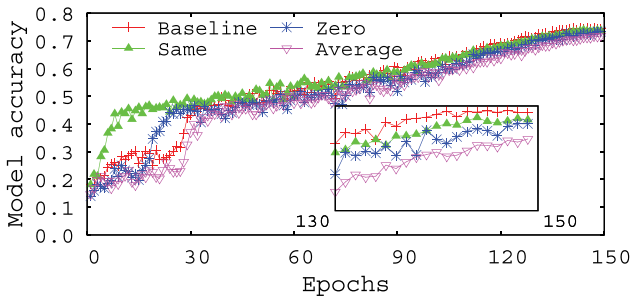


Fig. 2. Impact of different imputation policies on the model accuracy (*ACC*).

matrices. This reduces the workload. The difference between the estimation and the true value can be amortized in *all-reduce*. Specifically, given a straggler, we construct new matrices by pruning ($\mathbb{H}\cdot\gamma$) columns from the normal input X and weight W_i^A. The remaining columns are concatenated as pruned input $\overline{X}$ and weight $\overline{W}_i^A$, so as to invoke the existing matrix operator for parallel computations. These pruned matrices are only temporarily used to reduce workloads by γ, without any impact on original matrices. In Fig. 1, the 2nd column is pruned and the workload is reduced by $\frac{1}{3}$ where the element at $Y_i(1,1)$ with true value 32 is estimated as 22.

Notably, X and W_i^A should prune the same columns. We enforce this coordination because the ith column vector in W_i^A indicates the connection weights with respect to the ith column input neurons in X. Once the former is pruned, the latter does not make sense and then can be also pruned.

For BP, we give another pruning policy. In (2), ∇W_i^A is directly used to refine W_i^A in the current layer; ∇X_i is involved in refining W_i^B in the previous layer. Both have significant impacts on parameters. If we still provide all matrix elements but everyone is inaccurate, the accuracy penalty can be large. Thus, our new policy is to preserve partial but exact elements, and prune others for workload reduction.

Fig. 1 uses $\nabla W_i^A=(\nabla Y_i)^T*X$ as an example. Resembling FP, X can be pruned based on γ. We state that FP and BP are run at different times. As a result, even for the same layer, it might suffer from different waiting costs in FP and BP, since the straggling degree dynamically involves. Thus, the γ settings as well as the specifically selected pruned data, need to be calculated independently. Here $\gamma=\frac{2}{3}$ in BP where the 2nd and the 3rd columns are pruned. That is different from $\frac{1}{3}$ in FP where only the 2nd column is pruned.

Recovering for Consistency Constraint: (1) and (2) tell us that matrices are used across layers and between FP-BP phases. There exists a strict dependency constraint on their dimensions. After pruning, the output $\overline{Y}_i$ in FP has the same shape size as the unpruned version, which guarantees the normal computations in subsequent layers. However, in BP, the size of $\nabla\overline{W}_i^A$ changes if X is pruned. Worse, $\nabla\overline{W}_i^A$ and $\overline{W}_i^A$ might have different dimensions since they are pruned independently, yielding the matching error during parameter refinement. Similarly, a normal-sized ∇X_i is required in matrix multiplication but the next linear layer might not guarantee it if pruning happens.

The solution is to add missing columns in $\nabla\overline{W}_i^A$, to get normal-sized $\nabla\widehat{W}_i^A$. We maintain a lightweight lineage with a series of three tuples <*layer_id*, *matrix_id*, P>, where P is a set of indexes related to pruned columns. Then we can exactly match gradients with weights for correct refinement. Another issue is what value should be imputed into missing dimensions. Many different imputation policies can be used. We can use the same values calculated in previous iteration (*Same*), the average from unpruned dimensions in the current iteration (*Average*), and the uniform zero value (*Zero*). We test the three policies by running the ViT foundation model with one billion parameters on Colossal-AI.[2] In addition, we set $\gamma=0.5$. In Fig. 2, We clearly see that *Same* has the best accuracy but at expense of expensive storage costs. *Zero* beats *Average*. We guess the reason is that such a setting minimizes the impact of missing dimensions. Notably, during the early training phase (0–30 epochs), both *Zero* and *Same* outperform the loss-free *Baseline*. This can be attributed to the fact that both imputation strategies encourage the model to partially ignore newly learned knowledge within the current epoch. As a result, the model quickly converges to a local optimum. However, it subsequently makes a long detour to escape this suboptimal region to pursue a more globally optimal solution. Overall, our final choice is thereby the compromised *Zero* which balances space complexity and accuracy.

Determining Pruning Ratio: A large γ can significantly reduce workloads and then improve the heterogeneous tolerance ability. However, many estimations and imputed zeros lead to

[2]The detailed settings are given in Section V-B.

a big accuracy loss. To seek an optimal setting, we compare the runtime T_i^j of task-i with the average of all $\mathbb{T}$ tasks T_{avg}^j, at the jth iteration. If $T_i^j > T_{avg}^j$, task-i becomes a straggler, and then uniformly calculates γ_i^j for every transformer layer. The idea is that the aggregated workload savings should offset not only the runtime gap $(T_i^j - T_{avg}^j)$, but also the additional pruning latency. The latter includes the static overhead Ω_1 when allocating memory for pruned sub-matrices, and the dynamic extracting/concatenating overhead $\Omega_2()$ related to all preserved data $\sum(1-\gamma_i^j)\cdot L_x$ from each matrix with L_x dimensions. (3) shows the calculation of γ_i^j. M_i^j is the runtime of all matrix multiplications. Here, T_i^j, M_i^j and T_{avg}^j are available at the end of an iteration. Ω_1 and Ω_2 can be tested by running a mini-job in advance and then used as prior knowledge.

$$\gamma_i^j \cdot M_i^j = T_i^j - T_{avg}^j - \Omega_1 - \Omega_2 \left(\sum \left(1-\gamma_i^j\right)\cdot L_x\right) \quad (3)$$

B. Priority Pruning

We next study what should be pruned. Blindly and randomly selected pruning data cannot control the impact on the direction and magnitude of gradient descent during training, especially for a large ratio γ in heavily heterogeneous environments. Below, we outline our priority technique.

Priority Pruning: As reported in the literature on priority training non-transformer models [22], weight parameters with small gradient variations typically have a marginal impact on the subsequent training rounds. Our tests reveal that this holds true for transformers. Moreover, the trend of gradient changes is roughly identical to that of weight parameter changes. Inspired by those, we dynamically prioritize weight dimensions so that the one with small variation can be pruned in high probability. Given the weight matrix with L columns, we then establish a priority list *pri_list* to maintain indexes of yet-to-be-pruned dimensions, with size $L_{pri} = L\cdot\gamma$. Also, a *w_var_list* is required to keep track of weight changes where the ith element is the average change of the ith column, after the jth update of statistics.

Incremental Priority Update: Note that once the ith weight column is pruned due to its small variation, the imputed zero values clearly weaken the refinement impact. That, in turn, yields a small variation and hence enforces this column to be frequently pruned. It is equivalent to permanent modification of the model structure, yielding heavy accuracy degradation. We terminate this endless loop by incrementally updating statistics. In particular, at the end of each epoch, elements in *w_var_list* related to pruned columns will be preserved. Others are normally updated. As refinement proceeds, weight elements generally converge to the local optimum. Thus, the variation of weights will definitely become small and then the corresponding dimensions can be selected for pruning.

Differentiated Pruning Ratios: Recall that in (3), we give a uniform pruning ratio for all layers. However, the numerical values of *weight* and *input* matrices can be very different across layers. That motivates us to differentiate pruning ratios γ_k^j specific to the kth layer. Now γ_k^j is solely determined by the layer-based weight variation δ_k^j. More specifically, the ith column residing on layer-k is added into the pruning candidate set if $\delta_{ik}^j < \theta = N_{iter}\cdot\theta_{iter}$. Here the threshold θ is computed by the number of iterations N_{iter} within an epoch and a micro-threshold θ_{iter}. The latter is set as 10^{-3} by default. By the candidate set size, we can immediately infer γ_k^j. But the de facto pruning ratio should be equal to $\max\{\gamma_k^j, \alpha\cdot\gamma^j\}$, to guarantee that we can satisfy the requirement of tolerating heterogeneity. Here the decay factor α is set as 0.8 by default to partially adjust pruning budgets among layers. That is, layers with large variations can be pruned fewer dimensions; while others with small variations will be pruned more.

Algorithm 1: ZERO-Resizing Execution (t-th epoch).

Input: α, θ, T^t, e, N_{layer}, N_{iter}, L, R, $weight^t = [w_1^t, w_2^t, ..., w_L^t]$

1: T_{avg}^t = all-reduce$(T^t)/e$
2: Compute γ^t by (3)
3: **for** $k = 1$ to N_{layer} **do**
4: $\quad\delta_k^t = \cup_{i=1}^{L_k} \sum_{j=1}^{R_k} |w_{ji}^t - w_{ji}^{t-1}|/R_k$
5: $\quad$sort δ_{ik}^t in descending order as $w\_var\_list^t$
6: $\quad$**for** $index$ not in $pri\_list_k^{t-1}$ **do**
7: $\quad\quad w\_var\_list_{index}^t = w\_var\_list_{index}^{t-1}$
8: $\quad$**end for**
9: $\quad L_{uni} = \sum_{i=1}^{L_k} \delta_{ik}^t > \theta$
10: $\quad\gamma_k^t = (1 - L_{uni}/L_k)$
11: $\quad\gamma_k^t = \max\{\gamma_k^t, \alpha\cdot\gamma^t\}$
12: $\quad L_{pri} = L_k * (1 - \gamma_k^t)$
13: $\quad pri\_list_k^t$ = top-L_{pri}SelectIndex(w_var_list)
14: $\quad$ascendSort$(pri_list_k^t)$
15: **end for**
16: **for** $j = 1$ to N_{iter} **do**
17: $\quad$**for** $k = 1$ to N_{layer} **do**
18: $\quad\quad$resize matrix during FP
19: $\quad\quad$resize and impute matrix during BP
20: $\quad\quad$update $weight$
21: $\quad$**end for**
22: **end for**

We finally introduces the process of ZERO-resizing. At the tth epoch in training, it first calculates the minimum pruning ratio γ^t adapted to the heterogeneity degree using (3). It then runs the differential pruning scheme to personalize ratios γ_k^t for every layer, constructing a priority list $pri\_list_k^t$. Afterwards, resizing operations are carried out using $pri\_list_k^t$ during forward and backward propagations.

Algorithm 1 introduces the process of ZERO-resizing for the straggler in the server. Here, R represents the row length of the parameter matrix, and L represents the column length of the parameter matrix. During the training process of the tth epoch, the first step involves calculating the minimum pruning ratio γ^t adapted to the heterogeneity degree using (3) (lines 1-2). Subsequently, we employ a differential pruning scheme to compute different pruning ratios γ_k^t for each layer of the transformer, constructing a priority list $pri\_list_k^t$ based on these ratios (lines 3-15). During forward and backward propagations, temporary resizing and imputation of matrices are carried out using $pri\_list_k^t$ to reduce computation workloads (lines 16-22).

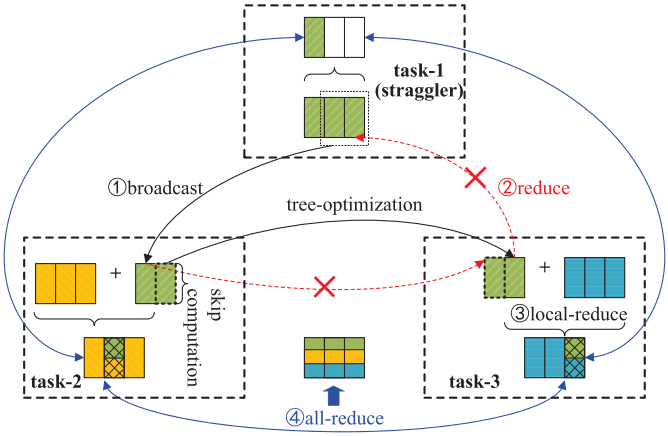


Fig. 3. Illustration of the sending-collecting migration.

IV. SEMI-MIGRATION: A HYBRID SOLUTION

This section presents a hybrid balancing solution SEMI-migration on top of the already designed priority resizing technique and a new lightweight migration scheme.

A. Lightweight Workload Migration

Matrix pruning comes with the drawback of accuracy loss, even with priority optimization. Dynamically re-balancing workloads is a classic solution without any accuracy penalty. However, the high cost of data migration becomes an obstacle. We thereby focus on lightweight optimizations.

Once a straggler task-i is detected, we can use a similar method as (3) to calculate how many columns should be migrated. Such data will be evenly distributed across other n^j normal tasks. But different from (3), now the runtime latency consists of transferring data from the straggler and processing immigrated data on normal tasks. The costs $\Phi_1()$ and $\Phi_2()$ are both related to the immigrated $\sum \gamma_i^j L_x$ workloads. (4) shows how to calculate γ_i^j.

$$\gamma_i^j \cdot M_i^j = T_i^j - T_{avg}^j - \Phi_1\left(\sum \gamma_i^j L_x\right) - \Phi_2\left(\frac{\sum \gamma_i^j L_x}{n^j}\right) \tag{4}$$

Since the straggling phenomena dynamically changes, the migration process is temporary. The straggler (sender) sends data to the target (receiver) and then collects results from the latter. That heavily burdens tasks, especially for the straggling sender. Selecting reasonable communication primitives becomes critical to reducing the cost $\Phi_1()$.

Our combination selection is *broadcast-reduce* where migration costs are amortized by normal tasks. Specifically, compared with the peer-to-peer *scatter*, *broadcast* has a built-in tree-based optimization, which enables other normal tasks already received data as new senders to continuously transfer data. That addresses the single-point communication bottleneck problem for the already slow straggler. For *reduce* and *gather*, we have the same analysis where the former can further amortize the abstract summation costs. Fig. 3 uses three tasks as an example to illustrate the two procedures ① and ②. The straggler task-1 broadcasts two columns to other normal tasks, but each of the latter computes only one column and skips computations related to other columns. task-2 and task-3, respectively, play the roles of new sender and collector for better scalability.

Further, we merge ②*reduce* with the normally executed ④*all-reduce* to further optimize migration costs. Fig. 3 shows the result of migrated workloads is collected by ②. But right after that, the built-in ④ is naturally performed in FP or BP. Our improvement is to delay ② and transform it from a global behavior into a local operation. That is, once a task completes its own computations and the newly assigned workloads, it directly accumulates the results of the two parts via a *local-reduce*. For example, task-3 adds migrated results into the 3rd column in its own output. These results can be correctly collected accompanied by ④.

B. A Hybrid Balancing Solution

Lightweight migration inevitably has runtime latency. By contrast, the resizing technique has prominent time and space complexities, but the accuracy degrades even with the help of priority pruning. Especially when the heavy heterogeneous phenomena happen, their drawbacks are exacerbated due to the large volume of migrated data and the significant number of imputed zeros. Thus, a smart choice is to build a hybrid solution on top of the two approaches.

The real-world heterogeneous environment is complex because the number of stragglers and the straggling skewness can be very different. We distinguish two scenarios. One involves a heavy straggler whose speed is far away from others'. The other involves many stragglers with different straggling degrees. We give our combination policies.

A Heavy Single Straggler: There exists the runtime gap $(T_s^j - T_{avg}^j)$ between the straggler task-s and other $(\mathbb{T}-1)$ normal tasks. The gap is generated by the workloads $\mathbb{W}_s = \sum \gamma_s^j \cdot L_x = \sum \frac{(T_s^j - T_{avg}^j) L_x}{M_s^j}$. We migrate some workloads to partially reduce the speed difference, with slight runtime latency. Then, by resizing, the straggler with emigrated data can easily catch up with fast tasks with immigrated data, with a slight accuracy penalty. As shown in (5), we determine the assignment configuration β to balance the resulting additional runtime latency for the straggler (Ω_1 & Ω_2) and normal tasks (Φ_1 & Φ_2). The goal is to avoid generating new waiting waste after using the two approaches.

$$\Omega_1 + \Omega_2\left((1-\beta)\cdot \mathbb{W}_s\right) = \Phi_1\left(\beta \mathbb{W}_s\right) + \Phi_2\left(\frac{\beta \mathbb{W}_s}{\mathbb{T}-1}\right) \tag{5}$$

Multiple Stragglers with Different Straggling Degrees: When multiple stragglers are present, each can be processed individually using (5). Given that resizing typically incurs lower latency than migration, a substantial portion of the workloads in $\mathbb{W}_s$ is handled through resizing. However, this leads to the accumulation of a large number of pruned elements across multiple stragglers, which can consequently result in significant accuracy degradation. To address this, we introduce a policy that employs a pre-defined threshold η as a global pruning ratio budget to mitigate the negative impact on accuracy. Furthermore, we strive to allocate this limited budget to as many modest straggling tasks

as possible. Such a greedy allocation strategy is motivated by two key considerations. First, each modest straggler requires only a limited number of elements to be pruned, and the abundance of preserved elements in these tasks helps to tolerate the pruning impact effectively. Second, by restricting migration to only a few severe stragglers, the overhead associated with establishing communication connections remains limited.

More specifically, we begin by sorting all stragglers in descending order based on their straggling degree. The reverse-top stragglers are then assigned to undergo resizing according to (3), while the remaining stragglers are processed via migration as defined in (4). A key challenge in this process is determining the exact boundary for the reverse-top straggler group. To address this, we traverse the sorted list of S stragglers in reverse order, computing the pruning ratio γ_i for each straggler task-i. The traversal terminates once the cumulative number of pruned elements just exceeds the predefined pruning budget, i.e.,

$$\arg \min_{i \in [1,S]} \sum \gamma_i \cdot \mathbb{M}_i > \eta \cdot \mathbb{M}$$

, where $\mathbb{M}_i$ denotes the number of matrix elements in task-i, and $\mathbb{M}$ represents the total number of elements across all tasks. In Section V-C, we empirically demonstrate that setting $\eta = 0.15$ serves as a robust and well-balanced configuration.

V. EXPERIMENTS

We implement our proposals on the new platform Colossal-AI [14] tailored for training foundation models. The code is available in https://github.com/fearless1007/ZERO.

A. Experimental Setups

Compared Solutions: The first competitor is the 1D tensor parallelism (TP) provided by Colossal-AI.[3] It overlaps computations with communications within each layer to mitigate the impact of the heterogeneous compute power [13]. We call it as *OverL*. Our proposals include the basic Resizing Matrix technique *RM* and its Priority variant *RM-Pri*, the Lightweight Migration *LM* and the hybrid solution *SEMI*. Besides, since this paper is the first attempt at heterogeneous TP, we build another competitor by borrowing ideas tailored for heterogeneous data parallelism. It is Flexible Synchronous Parallel *FSP* [23] where the volume of input samples is customized across tasks to tune workloads on demand. Now, similar with *RM*, missing dimensions are imputed by zeros to avoid failures under our TP scenario.

Benchmark Models and Datasets: Our benchmark foundation models include ViT [2] and OPT [24], respectively for image processing and NLP. We customize $\mathbb{D}$ and $\mathbb{H}$ to build ViT-1B with the architecture (24, 2048) and 1.2 billion parameters; ViT-3B with (24, 3072) and 2.7 billion parameters; and OPT-7B with (32, 4096) and 6.7 billion parameters. Table I shows the datasets, including CIFAR [25] and ImageNet [26] for ViT, and XSum [27] for OPT. For the ImageNet and XSum datasets, we construct representative subsets through random sampling. These reduced-scale datasets make multi-epoch training computationally tractable and thereby enable the efficient collection of sufficient statistics for reliable performance analysis.

TABLE I
DESCRIPTION OF DATASETS

Models	*DataSets*	*#Samples*	*#Dimensions*	*#Classes*
ViT	CIFAR10	60,000	32×32	10
ViT	CIFAR100	60,000	32×32	100
ViT	ImageNet	13,500	224×224	10
OPT	XSum	10,000	—	—

Hardwares and Simulation Testbed: All tests related to ViT are conducted on a GN10Xp.20XLARGE320 server in Tencent Cloud. It is equipped with 8 NVIDIA Tesla V100 GPUs, each with 32 GB HBM2 and offering 15.7 TFLOPS compute power. For OPT-7B, we employ a local workstation with 4 NVIDIA RTX A6000 GPUs, each with 48GB GDDR6 and 38.7 TFLOPS power. Resources are allocated in the best-effort manner. By default, we run 8 tasks for the training jobs of ViT and OPT, and fairly schedule them onto available GPUs.[4] We build the **real-world heterogeneous** environment by submitting another job with only one task to train a small ViT-base model. Further, existing studies have concluded that it is hard to accurately distinguish massive and dependent straggling factors, even though they indeed exist in practice [7], [28]. We thereby also build the **simulated heterogeneous** testbed by injecting sleeping operations to suspend threads, for ξ tasks selected as stragglers. In particular, we quantify the straggling degree by χ. That is, the injected matrix multiplication is χ times slower than the normal version.

Implementations: Similar to big data processing systems like Hadoop, our framework employs a heartbeat mechanism for periodic task monitoring and straggler detection. The hyper-parameters Ω_1, Ω_2, Φ_1, and Φ_2 are updated at each epoch to maintain an accurate reflection of the dynamic training environment. These monitoring and updating procedures operate independently and asynchronously, ensuring no interference with the primary training process. Furthermore, in the *LM* method, we pipeline the data migration with training computations, effectively overlapping these phases to mitigate the latency introduced by migration.

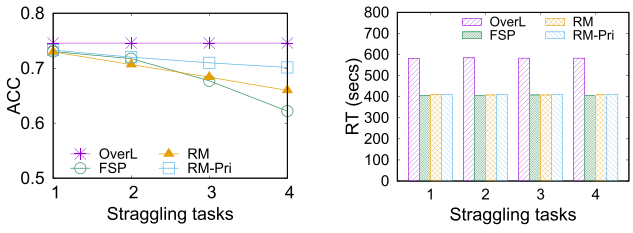
Others: Because we trade accuracy for efficiency, we are primarily concerned with the model accuracy *ACC* and the training runtime *RT*. We report *RT* as the averaged elapsed time of an epoch. We measure *ACC* by the *Top*-1 accuracy for ViT and *ROUGE*-1 for OPT.

B. Evaluation on Resizing Matrices

We explore the scalability by varying ξ and χ. Every task plays as the straggler in a round-robin manner, to simulate the dynamic change of heterogeneity. Figs. 4 –7 plots *ACC* and *RT* versus ξ, with χ fixed as 2. All solutions except *OverL* guarantee that stragglers can roughly catch up with normal tasks, and hence *RT*

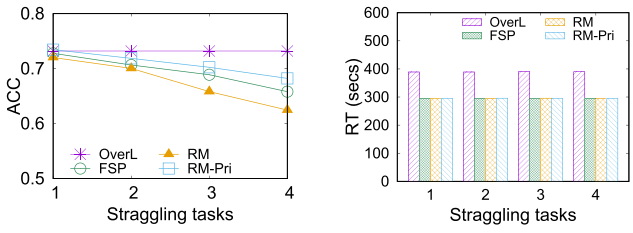
[3]Our techniques are also suitable for advanced 2D and 3D variants with specific network topology requirements of GPUs.

[4]There exist two tasks per A6000 GPU for OPT on the local workstation.



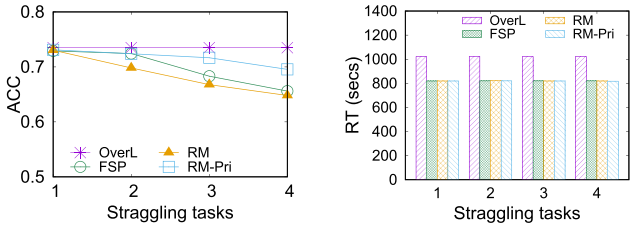
(a) Model accuracy (b) Training efficiency

Fig. 4. Scalability when varying ξ (ViT-1B, CIFAR10).



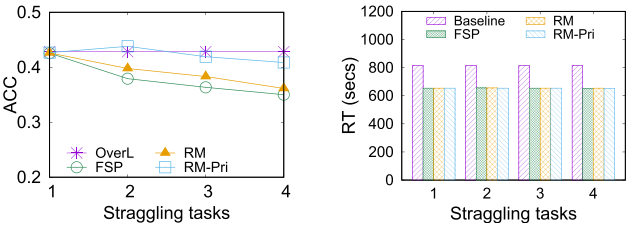
(a) Model accuracy (b) Training efficiency

Fig. 5. Scalability when varying ξ (ViT-1B, ImageNet).



(a) Model accuracy (b) Training efficiency

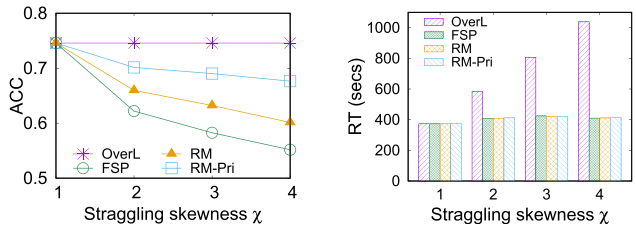
Fig. 6. Scalability when varying ξ (ViT-3B, CIFAR10).



(a) Model accuracy (b) Training efficiency

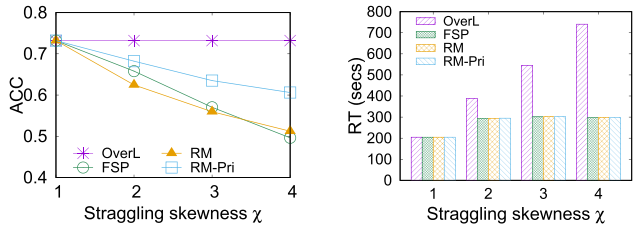
Fig. 7. Scalability when varying ξ (OPT-7B, XSum).

keeps steady. However, they have different accuracy penalties. For ViT, the accuracy loss of our *RM-Pri* increases from 1.3% to 4.4%. It beats *RM* (from 1.6% to 8.6%) and *FSP* (from 1.6% to 12.4%). That validates the effectiveness of priority pruning. For OPT, we observe better performance. *RM-Pri* achieves accuracy comparable to the loss-free baseline *OverL*. As illustrated in Fig. 7(a), the maximum accuracy degradation is only 1.9% when $\xi = 4$. In the case of $\xi = 2$, the accuracy even slightly surpasses that of *OverL*. We attribute this robustness to the fact that pruning less important model elements may help mitigate hallucination effects of the language model.



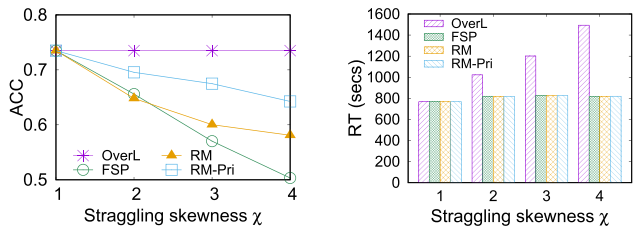
(a) Model accuracy (b) Training efficiency

Fig. 8. Scalability when varying χ (ViT-1B, CIFAR10).



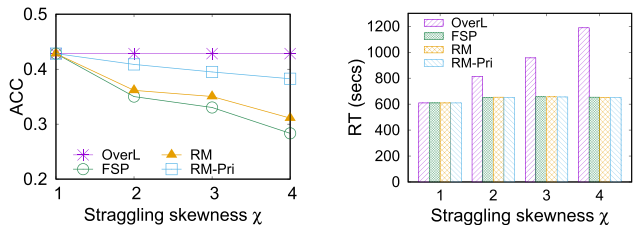
(a) Model accuracy (b) Training efficiency

Fig. 9. Scalability when varying χ (ViT-1B, ImageNet).



(a) Model accuracy (b) Training efficiency

Fig. 10. Scalability when varying χ (ViT-3B, CIFAR10).



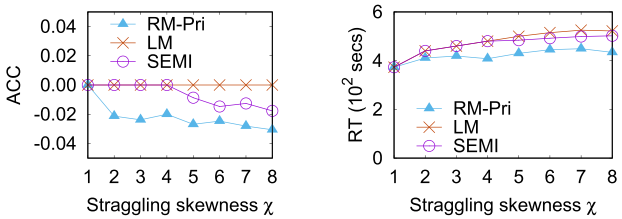
(a) Model accuracy (b) Training efficiency

Fig. 11. Scalability when varying χ (OPT-7B, XSum).

We observe the similar phenomenon in Figs. 8 –11, where χ varies from 1 to 4 but $\xi = 4$. For *OverL*, its *RT* linearly increases with χ, because the overall runtime is dominated by the slowest task, instead of how many tasks run slowly. By contrast, the other three methods have prominent *RT* performance with $<5\%$ runtime penalty on average. When $\chi = 4$, compared against *OverL*, *RM-Pri* reduces *RT* by 60.2%, with an acceptable 3.3% accuracy loss (see Fig. 11(a)). Compared to the better one between *RM* and *FSP*, it achieves a comparable runtime but reduces the accuracy loss by 67%.

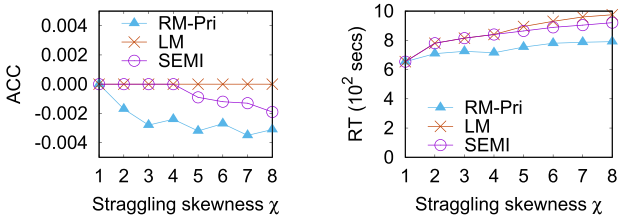
TABLE II
COMMUNICATION EFFICIENCY IN *LM* (SECS)

Primitives / γ	**0.00**	**0.25**	**0.50**	**0.75**	**1.00**
broadcast-reduce(1)	373	425	451	473	500
scatter-gather(1)	373	478	627	796	963
broadcast-reduce(4)	373	737	846	998	1,113
scatter-gather(4)	373	779	971	1,198	1,436



(a) Model accuracy (b) Training efficiency

Fig. 12. Scalability of *LM* and *SEMI* (one straggler, CIFAR10, ViT-1B).



(a) Model accuracy (b) Training efficiency

Fig. 13. Scalability of *LM* and *SEMI* (one straggler, XSum, OPT-7B).

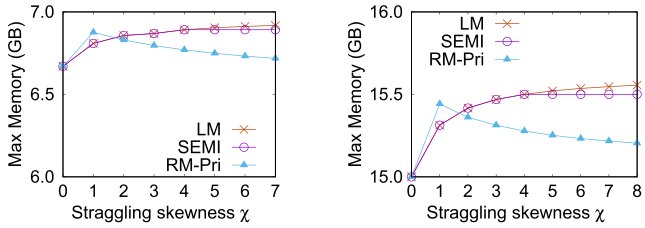
C. Evaluation on Data Migration

Now we test *LM* and *SEMI*. First of all, we analyze the communication efficiency of *broadcast-reduce* and *scatter-gather* in Table II. We migrate data measured by γ from 1 or 4 selected tasks to other tasks. All tests are run in homogeneous environments, for ViT-1B on CIFAR10. By the reported runtime per epoch, we see *broadcast-reduce* outperforms *scatter-gather*, due to the high-throughput implementation and our *reduce*-merging design.

Next, we give a detailed behavior analysis of complex scenarios. To clearly distinguish solutions in accuracy, we use *OverL*/*LM* as the standard marked with zero and then report the difference generated by each solution.

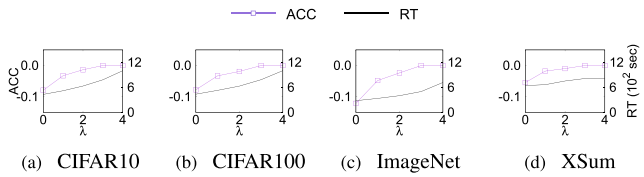
Figs. 12 and 13 plot *ACC* and *RT* against χ when there is only one straggler. Here *ACC* indicates the accuracy loss compared with the loss-free solution *OverL*. *LM* has the best *ACC* performance with zero loss. Further, benefiting from the asynchronously overlapped data migration, the increase of its runtime *RT* is a modest 40% (from 13%) relative to the non-straggling normal task. On the other hand, *RM-Pri* is characterized by the most significant accuracy loss, exhibiting only a 10% to 20% *RT* increase. *SEMI* makes a compromise between efficiency and accuracy.

Following the single-straggler scenario, Fig. 14 illustrates the peak memory footprint of the most memory-intensive task under varying straggling skewness χ. Overall, all three



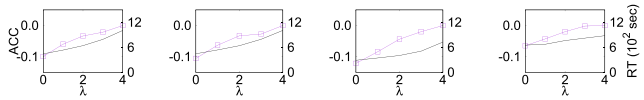
(a) ViT-1B (b) ViT-3B

Fig. 14. Impact of χ on the peak memory footprint (one straggler, CIFAR10).



(a) CIFAR10 (b) CIFAR100 (c) ImageNet (d) XSum

Fig. 15. Scalability of *SEMI* ($\xi = 4$ and χ settings are $\{4, 3, 2, 1\}$).

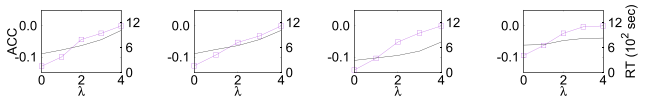


(a) CIFAR10 (b) CIFAR100 (c) ImageNet (d) XSum

Fig. 16. Scalability of *SEMI* ($\xi = 4$ and χ settings are $\{8, 6, 4, 2\}$)

strategies introduce acceptable memory overhead, with maximum increases of 2.6% for *RM-Pri* and 3.4% for *LM* and *SEMI* compared to the straggler-free baseline ($\chi = 0$). Under *RM-Pri*, the straggler becomes the most memory-intensive task due to the need to duplicate preserved matrices after pruning. A higher χ implies more elements are pruned, reducing the number of preserved elements. Consequently, the memory footprint gradually decreases as χ increases. In contrast, under *LM*, the volume of migrated data scales proportionally with χ. As a result, the normal non-straggling task that receives the migrated data becomes memory-intensive, exhibiting a corresponding proportional increase in memory usage. The behavior of *SEMI* is more complex. As χ increases, the method initially employs *LM* owing to its low runtime overhead, resulting in a proportional rise in memory footprint similar to that of *LM*. Beyond $\chi = 4$, however, data migration is discontinued due to excessive latency, and *RM-Pri* is activated instead. At this stage, the high χ value under *RM-Pri* introduces only minimal memory overhead on the straggler. The normal task receiving migrated data remains the most memory-intensive, yet its peak memory footprint stabilizes since data migration has stopped. In summary, the peak memory footprint of *SEMI* initially increases in a manner consistent with *LM*, then keeps stable after the transition point.

Figs. 15– 17 present a scalability analysis of our *SEMI* solution under a scenario where half of the tasks are stragglers ($\xi = 4$) with varying straggling degrees (as controlled by χ). In each figure, subfigures (a)– (c) report results on ViT-1B, while subfigure (d) corresponds to OPT-7B. To determine a suitable value for the hyper-parameter η, we repeatedly execute *SEMI* while systematically varying the number of stragglers



(a) CIFAR10 (b) CIFAR100 (c) ImageNet (d) XSum

Fig. 17. Scalability of *SEMI* ($\xi = 4$ and χ settings are {16, 12, 8, 4}).

TABLE III
TESTS IN REAL ENVIRONMENTS (VIT-1B, CIFAR10, *RT*: SECS)

ViT-base	OverL		*FSP*		*RM-Pri*		*LM*		*SEMI*	
(𝔻, ℍ)	*RT*	*ACC*	*RT*	*ACC*	*RT*	*ACC*	*RT*	*ACC*	*RT*	*ACC*
w/o	373	74.58	378	74.58	379	74.58	375	74.58	375	74.58
(12, 768)	473	–	432	73.88	432	73.37	481	–	432	73.37
(12,1024)	506	–	433	72.86	434	74.29	495	–	434	74.29
(12,1408)	572	–	439	71.10	441	73.71	524	–	501	74.58
(12,1792)	635	–	452	71.65	456	72.94	539	–	510	74.58
(12,2048)	670	–	450	70.52	453	72.19	550	–	523	73.10

TABLE IV
TESTS IN REAL ENVIRONMENTS (VIT-1B, IMAGENET, *RT*: SECS)

ViT-base	OverL		*FSP*		*RM-Pri*		*LM*		*SEMI*	
(𝔻, ℍ)	*RT*	*ACC*	*RT*	*ACC*	*RT*	*ACC*	*RT*	*ACC*	*RT*	*ACC*
w/o	205	73.20	207	73.20	207	73.20	209	73.20	208	73.20
(12, 768)	263	–	251	72.16	251	72.66	271	–	251	72.66
(12,1024)	280	–	248	72.01	248	73.09	286	–	248	73.09
(12,1408)	311	–	260	71.39	261	72.59	301	–	261	72.59
(12,1792)	352	–	273	69.97	272	71.41	319	–	297	71.86
(12,2048)	379	–	299	69.62	298	71.03	327	–	309	72.03

processed by the *LM* strategy, denoted as λ. Like Figs. 12 and 13, here *ACC* indicates the accuracy degradation where zero is the best. Our experiments reveal that the degradation is significantly alleviated when λ does not exceed 1, 2, and 2 in Figs. 15, 16, and 17, respectively. Beyond these values, the effectiveness of the alleviation is found to be marginal, while the runtime latency increases rapidly. Notably, at these respective λ thresholds, the resulting pruning budget η consistently stabilizes around 15%. We thereby recommend $\eta = 0.15$ as a pre-defined general setting.

D. Real-world Heterogeneous Evaluation

We vary the number of parameters of ViT-base from 86 million (12,768) to 600 million (12,2048). We accordingly allocate more resources to the corresponding job/task. By contrast, resources allocated to the task of ViT-1B decrease, leading to the increase of skewness degree. Table III lists results on CIFAR-10. When ViT-base becomes larger and deeper, our *RM-Pri* scales well in both *RT* and *ACC*. It achieves a 32.39% reduction in runtime latency while incurring an accuracy loss of 2.39%. In comparison, the loss-free baseline *LM* achieves a more modest runtime improvement of approximately 18%. The *SEMI* method, by contrast, strikes a balance between efficiency and accuracy, delivering a 22% runtime improvement in exchange for an accuracy loss of 1.48%. We also conduct experiments on the ImageNet dataset, with the results summarized in Table IV. Observations from these results indicate a performance improvement consistent with that observed on CIFAR-10.

VI. RELATED WORKS

There is a flurry of efforts for the resulting straggling problem in heterogeneous environments. Some researchers have explored job scheduling strategies that allocate workloads to appropriate hardware resources [29], [30], [31]. These scheduling policies can dynamically adapt to changes in resource availability for newly arrived yet unscheduled jobs. However, once the initial scheduling decision is made, the assigned parallel tasks remain fixed throughout the entire training process. Among these approaches, Liu et al. propose a fine-grained scheduling method that differentiates among the computational demands of different neural network layers and assigns them to suitable hardware accordingly [31]. Nevertheless, this approach is less effective for transformer-based models, as their uniform architecture results in consistent workload patterns across all transformer modules.

Unlike heterogeneous-aware yet static scheduling policies, many researchers have focused on dynamically optimizing resource utilization during the training of individual jobs. Most of these works are tailored for data parallelism. Some attempt to relax consistency constraints. Towards this end, they confine the synchronization barrier to a subset of tasks running fast and others as stragglers are flexibly skipped [9], [32], [33], [34], which wastes compute power. Another alternative is stale synchronous parallel [6], [35], [36], or even asynchronous parallel without any barrier [37], [38]. A recent work distinguishes the training stages and then proposes a smart switching scheme on the synchronization frequency [10]. These works mitigate the straggling problem but generate different versions of parameters specific to tasks. The accumulated error might lead to divergence [23], especially for tensor parallelism with very frequent synchronizations. Besides, dynamic data migration as a classic solution can avoid any accuracy loss [28], [39], [40], but the key issue is to reduce migration costs. Existing optimizations include pre-replicating samples [7] and overlapping computations with migration. They are motivated by the fact that not all samples are used in a batch/iteration [23]. Unused samples thereby can be migrated in an asynchronous manner. We also follow the migration idea but abandon corresponding optimizations, because the former challenges the limited GPU memory capacity; while the latter does not work for tensor parallelism where all tensors are involved in computations. Instead, we give our own lightweight optimizations. Note that the partial-use feature of samples also motivates researchers to tune workloads on demand by customizing batch size settings for different tasks [8], [23], [41], [42], [43]. Under tensor parallelism, this design changes the size of output, yielding the all-reduce communication failure. We attempt to avoid the failure by imputing missing values, but as reported, the accuracy loss is incurred.

Recently workload balancing for model parallelism in heterogeneous environments has also gained increasing attention. Park et al. logically group tasks where pipeline parallelism is used within a group and data parallelism is used across groups [44]. At the very beginning of training, they differentiate the group size to react to heterogeneity. Other works focus on the skewed initial partitioning by considering different network bandwidths and compute powers [11], [12]. However, all of these techniques are static, that cannot cope with dynamic stragglers. We are also aware that overlapping computations and communications can improve training efficiency and partially eliminate the negative impact of heterogeneous compute power [13]. Colossal-AI

integrates this design, but the effectiveness is marginal based on our tests, because this optimization only works within a layer.

VII. Conclusion

This paper studies the straggling issue of tensor parallelism, which has recently emerged when training foundation models in multi-tenants heterogeneous environments. We propose to resize matrices involved in core tensor computations with priority enhancement, and data migration with lightweight optimizations. Both can dynamically balance workloads to reduce waiting costs, but they have different advantages in terms of efficiency and accuracy. We thereby design a hybrid solution by combining them, which achieves overall success, as validated in experiments.

References

[1] H. Touvron et al., "LLAMA 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.

[2] M. Dehghani et al., "Scaling vision transformers to 22 Billion parameters," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 7480–7512.

[3] M. Li et al., "Scaling distributed machine learning with the parameter server," in *Proc. USENIX Symp. Operating Syst. Des. Implementation*, 2014, pp. 583–598.

[4] Y. Huang et al., "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 103–112.

[5] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using model parallelism," 2019, *arXiv:1909.08053*.

[6] Q. Ho et al., "More effective distributed ML via a stale synchronous parallel parameter server," in *Proc. Adv. Neural Inf. Process. Syst.*, 2013, pp. 1223–1231.

[7] A. Harlap et al., "Addressing the straggler problem for iterative convergent parallel ML," in *Proc. 7th ACM Symp. Cloud Comput.*, 2016, pp. 98–111.

[8] C. Chen, Q. Weng, W. Wang, B. Li, and B. Li, "Fast distributed deep learning via worker-adaptive batch sizing," in *Proc. ACM Symp. Cloud Comput.*, 2018, pp. 521–521.

[9] H. Kim, C. Song, H. Lee, and H. Yu, "Addressing straggler problem through dynamic partial all-reduce for distributed deep learning in heterogeneous GPU clusters," in *Proc. IEEE Int. Conf. Consum. Electron*, 2023, pp. 1–6.

[10] Z. Shen et al., "ASHL: An adaptive multi-stage distributed deep learning training scheme for heterogeneous environments," *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 30–43, Jan. 2024.

[11] Y. Duan et al., "HPH: Hybrid parallelism on heterogeneous clusters for accelerating large-scale DNNs training," in *Proc. IEEE Int. Conf. Cluster Comput.*, 2022, pp. 313–323.

[12] D. Li, H. Wang, E. Xing, and H. Zhang, "AMP: Automatically finding model parallel strategies with heterogeneity awareness," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 6630–6639.

[13] S. Singh, P. Singhania, A. K. Ranjan, Z. Sating, and A. Bhatele, "A 4D hybrid algorithm to scale parallel training to thousands of GPUs," 2024, *arXiv:2305.13525*.

[14] S. Li et al., "Colossal-AI: A unified deep learning system for large-scale parallel training," in *Proc. 52nd Int. Conf. Parallel Process.*, 2023, pp. 766–775.

[15] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 8026–8037.

[16] "Meta ai supercomputer DGX system overview," Accessed: Jan. 24, 2022. [Online]. Available: https://blogs.nvidia.com/blog/meta-ai-supercomputer-dgx/

[17] "Amazon EC2 on-demand pricing," Accessed: Jul. 1, 2024. [Online]. Available: https://aws.amazon.com/ec2/pricing/on-demand/?nc1=h_ls

[18] Y. Bao, Y. Peng, and C. Wu, "Deep learning-based job placement in distributed machine learning clusters with heterogeneous workloads," *IEEE/ACM Trans. Netw.*, vol. 31, no. 2, pp. 634–647, 2023.

[19] Z. Yang, H. Wu, Y. Xu, Y. Wu, H. Zhong, and W. Zhang, "Hydra: Deadline-aware and efficiency-oriented scheduling for deep learning jobs on heterogeneous GPUs," *IEEE Trans. Comput.*, vol. 72, no. 8, pp. 2224–2236, Aug. 2023.

[20] "Tencent cloud - Resource management and scheduling overview," Accessed: Dec. 11, 2024. [Online]. Available: https://www.tencentcloud.com/document/product/457/59103

[21] "Tencent cloud - cloud native scheduling overview," Accessed: Dec. 21, 2024. [Online]. Available: https://www.tencentcloud.com/document/product/457/52078

[22] X. Sun et al., "Training simplification and model simplification for deep learning: A minimal effort back propagation method," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 2, pp. 374–387, Feb. 2020.

[23] Z. Wang et al., "FSP: Towards flexible synchronous parallel frameworks for distributed machine learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 2, pp. 687–703, Feb. 2023.

[24] S. Zhang et al., "OPT: Open pre-trained transformer language models," 2022, *arXiv:2205.01068*.

[25] A. Krizhevsky, "Cifar image dataset," Accessed: Apr. 8, 2009. [Online]. Available: https://www.cs.toronto.edu/kriz/cifar.html

[26] "ImageNet large scale visual recognition dataset," Accessed: Mar. 11, 2021. [Online]. Available: https://www.image-net.org/

[27] S. Narayan, S. B. Cohen, and M. Lapata, "Don't give me the details, just the summary! Topic-aware convolutional neural networks for extreme summarization," *ArXiv*, vol. abs/1808.08745, 2018.

[28] S. Li, J. Xue, Z. Yang, and Y. Dai, "Efficient distributed machine learning with trigger driven parallel training," in *Proc. IEEE Glob. Commun. Conf.*, 2016, pp. 1–6.

[29] Z. Mo, H. Xu, and C. Xu, "HEET: Accelerating elastic training in heterogeneous deep learning clusters," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2024, pp. 499–513.

[30] Y. Bi, Y. Xi, and C. Jing, "AISAW: An adaptive interference-aware scheduling algorithm for acceleration of deep learning workloads training on distributed heterogeneous systems," *Future Gener. Comput. Syst.*, vol. 166, 2025, Art. no. 107642.

[31] J. Liu et al., "Heterps: Distributed deep learning with reinforcement learning based scheduling in heterogeneous environments," *Future Gener. Comput. Syst.*, vol. 148, pp. 106–117, 2023.

[32] C. Xu, G. Neglia, and N. Sebastianelli, "Dynamic backup workers for parallel machine learning," *Comput. Netw.*, vol. 188, 2021, Art. no. 107846.

[33] H. Yu, Z. Zhu, X. Chen, Y. Cheng, Y. Hu, and X. Li, "Accelerating distributed training in heterogeneous clusters via a straggler-aware parameter server," in *Proc. IEEE 21st Int. Conf. High Perform. Comput. Commun., IEEE 17th Int. Conf. Smart City, IEEE 5th Int. Conf. Data Sci. Syst.*, 2019, pp. 200–207.

[34] Y. Wang, T. Geng, E. Silva, and J. Gaudiot, "Hierarchical heterogeneous cluster systems for scalable distributed deep learning," in *Proc. 27th IEEE Int. Symp. Real-Time Distrib. Comput.*, 2024, pp. 1–6.

[35] Q. Zhou et al., "Petrel: Heterogeneity-aware distributed deep learning via hybrid synchronization," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1030–1043, May2021.

[36] M. Zheng, D. Mao, L. Yang, Y. Wei, and Z. Hu, "DOSP: An optimal synchronization of parameter server for distributed machine learning," *J. Supercomputing*, vol. 78, no. 12, pp. 13865–13892, 2022.

[37] M. Li et al., "Scaling distributed machine learning with the parameter server," in *Proc. 11th USENIX Symp. Operating Syst. Des. Implementation*, 2014, pp. 583–598.

[38] S. Soori, B. Can, M. Gurbuzbalaban, and M. M. Dehnavi, "ASYNC: A cloud engine with asynchrony and history for distributed machine learning," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2020, pp. 429–439.

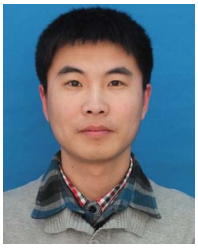
[39] U. A. Acar, A. Charguéraud, and M. Rainey, "Scheduling parallel programs by work stealing with private deques," in *Proc. 18th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, 2013, pp. 219–228.

[40] S. Wang, W. Chen, A. Pi, and X. Zhou, "Aggressive synchronization with partial processing for iterative ML jobs on clusters," in *Proc. 19th Int. Middleware Conf.*, 2018, pp. 253–265.

[41] R. Han, S. Li, X. Wang, C. H. Liu, G. Xin, and L. Y. Chen, "Accelerating gossip-based deep learning in heterogeneous edge computing platforms," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1591–1602, Jul. 2021.

[42] Q. Zhou et al., "Falcon: Addressing stragglers in heterogeneous parameter server via multiple parallelism," *IEEE Trans. Comput.*, vol. 70, no. 1, pp. 139–155, Jan. 2021.

[43] K. Liu, J. Wang, Z. Huang, and J. Pan, "Sampling-based multi-job placement for heterogeneous deep learning clusters," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 6, pp. 874–888, Jun. 2024.
[44] J. H. Park et al., "{HetPipe }: Enabling large { DNN} training on (whimpy) heterogeneous {GPU} clusters through integration of pipelined model parallelism and data parallelism," in *Proc. USENIX Annu. Tech. Conf.*, 2020, pp. 307–321.



Zhigang Wang received the PhD degree in computer software and theory from Northeastern University, China, in 2018. He is currently an associate professor with Guangzhou University. His research interests include distributed graph processing and machine learning. He is the China Computer Federation (CCF). He won CCF Outstanding Doctoral Dissertation Award (2018), ACM QINGDAO Rising Star Award (2019), and the second prize of CCF Natural Science Award (2023).

Xu Zhang received the bachelor's degree from the School of Computer Science and Engineering, Shandong University of Science and Technology, China, in 2022. He is currently working toward the master's degree with the School of Computer Science and Technology, Ocean University of China. His research interests include deep learning and parallel computing.

Ning Wang received the PhD degree in computer software and theory from Northeastern University, China, in 2017. She is currently an associate professor with Guangzhou University. She has been a visiting PhD student in Nanyang Technological University during 2014 to 2015. Her research interests include big data process, graph analysis, and differential privacy protection. She won ACM QINGDAO Rising Star Award (2021).

Chuanfei Xu received the BE degree from Shenyang University of Technology, in 2007, the ME and PhD degrees from Computer Science, Northeastern University, China, in 2009 and 2013, respectively. He is currently working as a researcher with the Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ). His research interests include spatial database management, parallel computing and natural language processing (NLP).

Yu Gu received the PhD degree in computer software and theory from Northeastern University, China, in 2010. Currently, he is a professor and the PhD supervisor with Northeastern University, China. His current research interests include big data analysis, distributed computation and graph data management. He is a senior member of CCF. He won the second prize of CCF Natural Science Award, in 2023.

Hui Lu received the PhD degree in electromagnetic field and microwave technology from the Beijing University of Posts and Telecommunications, Beijing, China, in 2010. He is currently a professor with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China. His research interests include deep learnig, cyberspace security and intelligent attack–defense.

Dawei Zhao received the PhD degree in cryptology from Beijing University of Posts and Telecommunications, Beijing, China, in 2014. He is currently a professor with Shandong Computer Science Center (National Supercomputer Center in Jinan), Jinan, China. His main research interests include network security, complex network, and epidemic spreading dynamics.

Zhihong Tian (Senior Member, IEEE) received the PhD degree in computer science and technology from the Harbin Institute of Technology, Harbin, China. He is currently a professor, and dean, with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China. He is also a part-time professor with Carlton University, Ottawa, Canada. Previously, he served in different academic and administrative positions with the Harbin Institute of Technology. He has authored more than 200 journal and conference papers in these areas. His research interests include computer networks and cyberspace security. He also served as a member, chair, and general chair of a number of international conferences. He is a senior member of the CCF.